

Final Project: Trip Planner

For this assignment you will implement a trip planning API (Application Programming Interface) that provides routing and searching services. This API could be used to power the backend of an application that helps tourists navigate a new city, to find places to eat or attractions to visit, for example.

Unlike previous homework assignments, the representation is not defined for you, nor have we specified which data structures and algorithms to use. Instead, you may choose from among the data structures you have implemented for previous assignments as well as any algorithms that we either implemented or saw in class. Selecting appropriate data structures and algorithms is up to you, and we expect you to approach these choices diligently and intelligently.

In `planner.rkt`, I've supplied a definition of the interface you need to implement, as well as auxiliary definitions discussed in this document.

Preamble

Most of the learning value of this assignment comes from figuring out the overall structure, moving pieces, and how they fit together, and less from the implementation of that plan. So you get the most out of this assignment, we **strongly discourage** discussing even general approach with your colleagues.

Similarly, taking other shortcuts will also sabotage your learning; it also goes without saying that shortcuts prohibited by the class's academic integrity policy (in the syllabus) will have very severe consequences.

Also, be warned that this project is significantly larger and longer than previous assignments. **We recommend you start as early as you can.**

How to Start

Read the entire handout fully and carefully until the end, take notes as you go, and process it in this first pass. If you haven't processed anything fully the first time, that's okay. You can fill in those gaps as you start to work on each component and through asking us clarifying questions.

Once you've read through the project requirements, section 5 contains advice on how to start. Read section 6 to understand how we can (and cannot) help you in office hours.

1 Problem Overview

Your trip planner will store map data representing three kinds of items and answer three kinds of queries about them.

1.1 Items

- A *position* has a latitude and a longitude, both numbers.
- A *road segment* has two endpoints, both positions.
- A *point-of-interest (POI)* has a position, a category (a string), and a name (a string). The name of a point-of-interest is unique across all points-of-interest, but a category may be shared by multiple points-of-interest. Each position can feature zero, one, or more POIs.

See figure 1 for an example of a map containing the three kinds of items.

We will make three assumptions about segments and positions:

1. All roads are two-way roads.
2. The length of a road segment is the standard Euclidian distance between its endpoints.
3. Points-of-interest can only be found at a road segment endpoint.

1.2 Queries

A complete trip planner must support three forms of queries:

locate-all Takes a point-of-interest category; returns the positions of all points-of-interest in the given category. The positions can returned be in any order you want, but the result should not include duplicates.

plan-route Takes a starting position (latitude and longitude) and the name of a point-of-interest; returns a shortest path from the starting position to the named point-of-interest. You can assume the starting position is at a road segment endpoint. Returns the empty list if the destination does not exist or is unreachable.

find-nearby Takes a starting position (latitude and longitude), a point-of-interest category, and a limit n ; returns the (up to) n points-of-interest in the given category nearest the starting position. You can assume the starting position is at a road segment endpoint. Resolve ties however you want, and order points-of-interest within the list in any order you want.

For some example queries and results see tables 1, 2, and 3. For specifics on the interface your code should conform to (i.e., precise input and output types), read on.

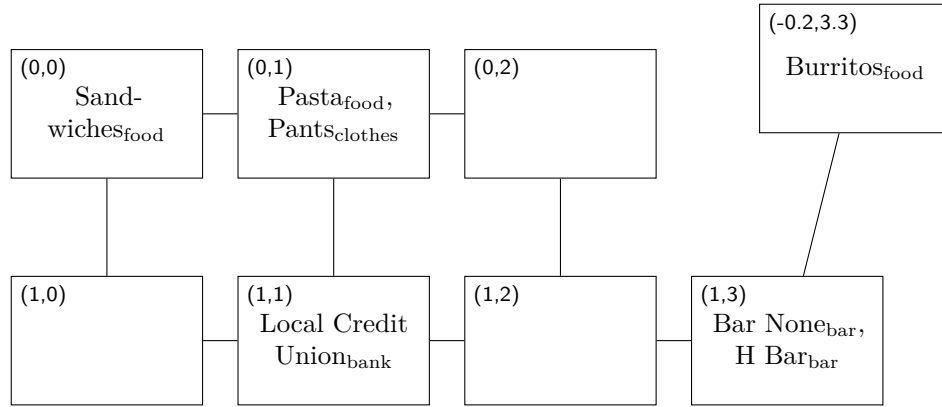


Figure 1: Example map with POIs labeled by latitude and longitude

Category	Result positions
food	(0,0), (0,1), (-0.2,3.3)
bank	(1,1)
bar	(1,3)
barber	<i>nothing</i>

Table 1: Example *locate-all* queries

Start	Name	Result path
(0,0)	Sandwiches	(0,0)
(0,1)	Sandwiches	(0,1)–(0,0)
(1,1)	Sandwiches	(1,1)–(1,0)–(0,0) <i>or</i> (1,1)–(0,1)–(0,0)
(1,1)	Burritos	(1,1)–(1,2)–(1,3)–(-0.2,3.3)
(1,1)	Sushi	<i>nothing</i>

Table 2: Example *plan-route* queries

Start	Category	n	Result POIs
(1,3)	food	1	Burritos
(0,2)	food	1	Pasta
(0,2)	food	2	Pasta, Sandwiches (in any order)
(0,2)	food	3	Pasta, Sandwiches, Burritos (in any order)
(0,2)	food	4	Pasta, Sandwiches, Burritos (in any order)
(0,2)	bar	1	Bar None <i>or</i> H Bar
(0,2)	bar	2	Bar None, H Bar (in any order)
(0,2)	bar	3	Bar None, H Bar (in any order)
(0,2)	school	5	<i>Nothing</i>

Table 3: Example *find-nearby* queries
(your implementation should return the full POIs, not just the names)

Representation	Contract	Purpose
num?	Lat?	latitude
num?	Lon?	longitude
str?	Cat?	POI category
str?	Name?	POI name
[Lat?, Lon?]	RawPos?	raw position
[Lat?, Lon?, Lat?, Lon?]	RawSeg?	raw road segment
[Lat?, Lon?, Cat?, Name?]	RawPOI?	raw POI
vector of road segments	VecC[RawSeg?]	input raw road segments
vector of POIs	VecC[RawPOI?]	input raw POIs
linked list of positions	ListC[RawPos?]	<i>locate-all</i> and <i>plan-route</i> result
linked list of POIs	ListC[RawPOI?]	<i>find-nearby</i> result

Table 4: Summary of vocabulary types

2 API Specification

The trip planner API is specified as a DSSL2 interface named `TRIP_PLANNER` (provided in the starter code), which you must implement as a class named `TripPlanner`.

The `TRIP_PLANNER` interface refers to a variety of “vocabulary types” that represent the three kinds of items discussed in section 1.1, as well as other types used by `TRIP_PLANNER` operations. All these types are summarized in table 4.

2.1 Basic Types

The basic vocabulary types include latitude and longitude (represented as numbers) and POI categories and names (represented as strings).

2.2 Raw Item Types

In the external API that your `TripPlanner` class must support, items are represented in “raw” form, i.e., as vectors (not structs) of basic types. Specifically:

- A raw position is represented as a 2-element vector containing a latitude and a longitude.
- A raw road segment is represented as a 4-element vector containing the latitude and longitude of one end position followed by the latitude and longitude of the other.
- A raw point-of-interest is represented as a 4-element vector containing the latitude and longitude of its position, then its category, and then its name.

These types are intended for communication between your `TripPlanner` class and the client; you will probably want to define richer representations for internal usage in your implementation.

2.3 Input Types

Your `TripPlanner` constructor must take two arguments:

- A vector of raw road segments.
- A vector of raw points-of-interest.

Both element types are as described in section 2.2.

In the starter code, we have provided a small example test that creates a `TripPlanner` instance with inputs in the correct format. You can use it to make sure your implementation conforms to the interface.

2.4 Output Types

The results for each of the three kinds of queries must be expressed in terms of the raw item types. Specifically:

- *locate-all* must return its result as a linked list (using the `cons` library) of raw positions. The elements of the list can be returned in any order, and each position may only appear once in the result.
- *plan-route* must also return a linked list (also using the `cons` library) of raw positions. The positions must be in the order in which they appear in the path from the source to the destination. For non-existent or unreachable destinations, *plan-route* must return the empty list.
- *find-nearby* must return a linked list (still using the `cons` library) of raw points-of-interest. The points-of-interest in the list can be in any order.

In the starter code, we have provided one small example test for each kind of query that checks that the output produced by your implementation are in the correct format. You can use them to make sure your implementation conforms to the interface.

3 Dependencies

Your solution is likely to rely on a number of different ADTs and data structures, including some which you've implemented as part of previous assignments.

As we did with homework 4, we're providing you with compiled versions of solutions, this time for homeworks 2, 3, 4, and 5. That way, everyone can start on firm footing regardless of the state of their previous homeworks.

As before, extract the `project-lib.zip` archive in the same directory as your `planner.rkt` file and import the files you need. The file names and interfaces are identical to the ones used for these assignments.

For example, `import 'project-lib/dictionaries.rkt'` would import a dictionary library whose interface is the one from homework 3. For reference, the graph representation used by our `graph.rkt` is an adjacency matrix.

Note: Be careful to recreate the exact directory structure described above and use the exact import syntax described above. Otherwise your code won't work, either when you try to run it yourself, or when we try to grade it.

Note: The compiled files we provide are specific to Racket version 8.16; if you use any other version (including a more recent one!), they will not work.

Alternatively, you can use your own homework solutions (e.g., if they are better suited to the task at hand). Because you will only turn in your `planner.rkt` file, you will need to copy over to it any homework code you wish to use.

You may also import modules from the DSSL2 standard library, such as the `cons` and `sbox_hash` libraries.

Any of the code I posted to Canvas alongside lectures is also fair game. You'll just have to copy their implementation to your `planner.rkt` file. And be sure to acknowledge your sources.

4 Design Documents

We expect you to consider the design of your trip planner carefully. To assess this, you will need to document various aspects of your design along the way. Specifically, for *each* of the three project deadlines, you will need to submit a design document.

4.1 Entity-Relationship Diagram

We expect you to plan out the design of your trip planner before you start implementing it; just like we did during Data Design week.

The first design document you will produce is an entity-relationship diagram—like the one we saw in class—outlining the different pieces of your trip planner. This deliverable will be due alongside the *first* deadline for your project program.

We will be looking for a *clear* plan that shows you have thought about how to approach the problem. That plan does not need to be bullet-proof or complete, but it needs to be *plausible* and *detailed* enough to make sense.

Note: we are especially looking for you to *plan ahead*! I.e., to try to predict what you may need later on; just like we did in class. Diagrams which describe only elements which are provided as inputs to the trip planner will not receive credit.

As mentioned in class, we will not be sticklers about the specifics of notation, and hand-drawn diagrams are fine. We also understand that, at that point of the process, you may not have a fully working trip planner, and that your design may continue to evolve until the final deadline; that's ok too, just show us the current state of your design.

In general, we're aiming to understand your planning and thought process. We are not looking for a specific design or for any particular elements; just for a clear and reasonable plan.

Note: Entities should be concepts from the problem domain, not abstract data types. I.e., if you use a queue to represent a checkout line at the grocery store, “checkout line” is the entity, not queue.

4.2 Ethical Review

We expect you to reflect critically on how the programs you write could be used, and in particular how they could be misused.

The second design document you will produce is an ethical review, exploring the implications of some hypothetical changes one could make to your trip planner. This deliverable will be due alongside the *second* project deadline.

Scenarios

1. Suppose your boss asked you to make businesses rank higher in `find_nearby`, if they provide discounts to trip planner users.
2. Suppose your boss asked you to add a feature to “shadow ban”¹ some businesses from the trip planner. If your company shadow bans a business, the owner of that business will see it in query results as normal, but it would not appear in the results for other users. I.e., the business will be “banned” for normal users, but the owner of the business would not notice.
3. As currently imagined, the trip planner stores a few pieces of information about each business it knows about: its name, its category, and its location. But one could imagine storing more information as well: e.g., year established, etc.
Choose one example kind of information we could **also** store about businesses in the trip planner and display to trip planner users. (Your example should be something other than the given example of “year established”.)

For each scenario, write a short paragraph that discusses:

1. What positive effects could making this change have?
2. What negative effects could making this change have?
3. Would you agree to implement it, and why?

¹https://en.wikipedia.org/wiki/Shadow_banning

As with the rest of the project, there are no clear-cut right or wrong answers. We are instead looking for *thoughtful*, *cohesive*, and *relevant* answers. Also, be *concrete*: for example, when discussing positive/negative impacts, a well-explained concrete example—e.g., person X had bad outcome Y because Z happened—can be an effective answer.

Reminder: these changes are hypothetical; do not include them in the trip planner you turn in!

4.3 Analysis of Alternatives

We expect you to put care and thought into your choices of ADTs, data structures, and algorithms. The third design document you will produce is a brief report explaining your decisions, in particular comparing them to alternatives and providing a rationale for why you made specific choices. This deliverable will be due alongside the *third and final* deadline for your project program.

For each use of an ADT in your program, list the following:

- What role does it play in solving the problem?
- What data structure did you pick for it?
- Why did you pick that data structure over other choices?

Each of these should be no more than a short paragraph. For conciseness and legibility, please format as a series of bullet points:

- *ADT (e.g., Queue #1)*
 - *role (1-2 sentences)*
 - *which data structure (e.g., linked list queue)*
 - *why this data structure and not another (1 short paragraph)*
- *ADT (e.g., Queue #2)*
 - *role (1-2 sentences)*
 - *which data structure (e.g., ring buffer)*
 - *why this data structure and not another (1 short paragraph)*
- ...

You don't need to describe intermediate structs, or intermediate arrays that are not meaningful sequences in their own right. But when in doubt, mention it.

Then, for each non-trivial algorithm in your program, explain the following, using the same format as for the ADT portion of the analysis:

- What role does it play in solving the problem?
- Why did you pick that algorithm over other choices?

By non-trivial algorithm, we mean an algorithm that is sophisticated enough to be worth discussing in its own right. For example, merge sort is non-trivial, but looping over an array to find the largest element is not. But when in doubt, mention it.

Note: Unlike self-evaluations for previous assignments or your entity-relationship diagram, your analysis of alternatives should describe your solution as it stands in your *final* submission.

When evaluating your analysis, we will be specifically looking for:

- **Justifications:** your choices of data structures and algorithms must be well-motivated, with clear pros and cons and comparisons to alternatives. You may find it helpful to refer back to the first Data Design lecture for discussion of possible decision criteria.
- **Completeness:** your analysis must describe all the elements required for a correct trip planner. The precise number and makeup of these elements will necessarily differ between solutions, however, and we will take this into account.
- **Clarity:** your writing must be clear, precise, and concise. Technical terminology must be used accurately, and technical arguments your analysis makes must be correct.

5 Advice

This project is a significant step up from earlier assignments both in terms of scale and in terms of challenge. To help you be successful, here are a few pieces of advice.

- **Start early!** Not only is there a lot to do, some of the design work can benefit from thinking about it on and off, while you do other things.
- Don't start writing code right away; planning out your design—as we did in the data design lectures—will save you time in the long run.
- You will want to implement *locate-all* (the easiest of the three queries) first, then try *plan-route*, and save *find-nearby* (the hardest of the three) for last. Of course, you'll also want to test each one thoroughly, to be confident in its correctness, before moving on to the next.
- When you get one of the queries working, *save a copy of your work*. That way, even if you end up breaking your program while working on another query, you'll at least be able to submit this older, partially working version and get feedback on it (and credit for it!) Better to submit a program that does less but does it successfully, than to submit a program that tries to do more but does not run at all.

- When designing your solution, you may be interested in using efficient, but somewhat complex algorithms or data structures. These can indeed be good choices, but getting them to work correctly may be difficult. If you find yourself struggling to get a complex algorithm or data structure to work, you may want to take a step back and (temporarily, at least) use a simpler equivalent instead. This simpler version can serve as a stopgap until you get the more complex solution to work, or as a fallback in case you don't get it working. Just be sure to explain your decision process in your Analysis of Alternatives.
- The project has a *very* significant impact on your base grade; see the syllabus. As such, we *highly* recommend you prioritize it.

6 Getting Help

This project is very open ended: we expect everyone to have different project plans and implementations; coming up with them is a major part of the learning for this assignment. If you plan to attend office hours to get help on the project, you will need to show us the preliminary planning you did; if we don't know what your plan is, we cannot help you make that plan a reality.

For this reason, *if you have not done your planning, we will not help you in office hours*. Instead, we will send you back to the drawing board; as we discussed during data design week, the planning process is an extremely important part of any project. Once you have a plan, though, we'll be more than happy to help!

For example, your plan can be similar to what we ask for in the “entity-relation diagram” document. Or it can be your notes from following (a prefix of) the first five steps of the data design recipe we saw in class. It does not have to be polished, nor does it have to be your final plan, but it should be clear enough that we can understand it.

7 Honor code

Every programming assignment you hand in must begin with the following definition (taken from the Provost's website;² see that for a more detailed explanation of these points):

```
let eight_principles = ["Know your rights.",
    "Acknowledge your sources.",
    "Protect your work.",
    "Avoid suspicion.",
    "Do your own work.",
    "Never falsify a record or permit another person to do so.",
    "Never fabricate data, citations, or experimental results.",
    "Always tell the truth when discussing your work with your instructor."]
```

²<http://www.northwestern.edu/provost/students/integrity/rules.html>

If the definition is not present, you receive no credit for the assignment.

Note: Be careful about formatting the above in your source code! Depending on your pdf reader, directly copy-pasting may not yield valid DSSL2 formatting. To avoid surprises, be sure to test your code *after* copying the above definition.

8 Grading

Please submit your completed version of `planner.rkt`, containing:

- a definition of the `TripPlanner` class,
- sufficient tests to be confident in your program's correctness,
- and the honor code.

Note: Be careful to submit a single `.rkt` file! If your submission imports anything besides `project-lib`, we will not be able to successfully run it; you must include any additional dependencies directly inside your one submission file.

Be sure to remove any leftover debugging printing code, comment out any code that would cause infinite loops, and make sure that your submission can run successfully. If your submission produces excessive output, loops infinitely, or crashes, we will not be able to give either you feedback on it or credit for it. **Please run your code one last time before submitting!**

Please also submit each of your design documents in `pdf` format to its (separate) Canvas assignment.

Functional Correctness

We will use six separate test suites to test your submission:

- **Basic locate:** construction of the example trip planner from figure 1, locate-all with at most one POI per location.
- **Advanced locate:** locate-all with multiple POIs per location, locate-all edge cases.
- **Basic route:** plan-route with one possible path.
- **Advanced route:** plan-route with multiple possible paths, plan-route edge cases, plan-route stress tests (to check for extreme inefficiencies).
- **Basic nearby:** find-nearby looking for the one closest relevant POI.
- **Advanced nearby:** find-nearby looking for multiple closest POIs, find-nearby edge cases, complex combinations of POIs and categories, find-nearby stress tests (to check for extreme inefficiencies).

To get credit for a test suite, your submission must pass *all* its tests.

The outcome your submission will earn will be determined as follows:

- **Got it:** passes all six test suites.
- **Almost there:** passes any four test suites.
- **On the way:** passes any two test suites.
- **Not yet:** does not achieve “on the way” requirements.
- **Cannot assess:** we could not successfully grade your submission.
- **Missing honor code:** your submission did not include the honor code.

In Extremis Fourth Try

As you can see in the syllabus, failure to earn at least **on the way** on the project results in base grade of **F**; and therefore a final grade of **F**.

Because this is a very significant impact, students who do not manage to reach that bar after the standard three tries will be given an *ad hoc, in extremis* fourth try in the hope they can avoid failing the class outright. For fairness to the students who do manage to get credit for the project, though, this fourth try will be capped to **on the way** and will carry a modest final grade penalty as well. Nonetheless, it remains a preferable alternative to an automatic **F**.

We will be in touch with students who end up in this situation to discuss details and specifics.

This does not apply to students who commit academic misconduct on the project: these students will not receive credit for the project, and will not be eligible for this *in extremis* fourth try; i.e., **students who commit academic misconduct on the final project will automatically fail the class**. This is a very poor decision; don't do that.

Non-Functional Correctness

There is no self-evaluation for the final project; we will therefore not be evaluating non-functional correctness for it. We nonetheless highly encourage you to continue applying the good habits you learned!

Design Documents

Each design document will receive a grade of either **Good**, **Attempted**, or **Insufficient**. If your document meets our expectations for all the criteria highlighted in its description above, we will consider it **Good**. If it falls short on any of them, but is otherwise a reasonable good faith attempt, we will consider it **Attempted**. Majorly incomplete or severely lacking submissions will be considered **Insufficient**.

To earn a positive (+1) modifier for your design documents, all three of your design documents must be **Good**.

To earn a neutral (0) modifier, at least one of your design documents must be **Good**, and the rest must be **Attempted**.

Failing to meet the criterion for a neutral modifier (i.e., one or more of your design documents are **Insufficient**, or all three are no better than **Attempted**) will result in a negative (-1) modifier.

In the event you misunderstand the requirements for the first or second design document—or make a minor mistake when writing it—you may spend a redo token to submit a corrected version; see details in the syllabus. This does **not** apply to the third and final design document; its deadline is already as late in the quarter as it can be, so there is no time left for resubmissions.

Reference

The following functionality is provided by each library in `project-lib.zip`. The behavior of each export is as described in the relevant homework handout; this reference is merely a convenience.

`project-lib/stack-queue.rkt`

```
interface STACK[T]:
  def push(self, element: T) -> NoneC
  def pop(self) -> T
  def empty?(self) -> bool?

# The QUEUE interface is exported by the `ring_buffer` library
# interface QUEUE[T]:
#   def enqueue(self, element: T) -> NoneC
#   def dequeue(self) -> T
#   def empty?(self) -> bool?

class ListStack[T] (STACK):
  def __init__(self)

class ListQueue[T] (QUEUE):
  def __init__(self)
```

`project-lib/dictionaries.rkt`

```
interface DICT[K, V]:
  def len(self) -> nat?
  def mem?(self, key: K) -> bool?
  def get(self, key: K) -> V
  def put(self, key: K, value: V) -> NoneC
  def del(self, key: K) -> NoneC

class AssociationList[K, V] (DICT):
  def __init__(self)

class HashTable[K, V] (DICT):
  def __init__(self, nbuckets: nat?, hash: FunC[AnyC, nat?])

def first_char_hasher(s: str?) -> int?
```

project-lib/graph.rkt

```

let Vertex? = nat?
let VertexList? = Cons.ListC[Vertex?]

let Weight? = AndC(num?, NotC(OrC(inf, -inf, nan)))
let OptWeight? = OrC(Weight?, NoneC)

struct WEdge:
  let u: Vertex?
  let v: Vertex?
  let w: Weight?
let WEdgeList? = Cons.ListC[WEdge?]

interface WUGRAPH:
  def n_vertices(self) -> nat?
  def n_edges (self) -> nat?
  def set_edge(self, u: Vertex?, v: Vertex?, w: OptWeight?) -> NoneC
  def get_edge(self, u: Vertex?, v: Vertex?) -> OptWeight?
  def get_adjacent(self, v: Vertex?) -> VertexList?
  def get_all_edges(self) -> WEdgeList?

class WUGraph (WUGRAPH):
  def __init__(self, size: nat?)

  def normalize_vertices(lst: Cons.list?) -> Cons.list?:
  def normalize_edges(lst: Cons.list?) -> Cons.list?:

  def dfs(graph: WUGRAPH!, start: Vertex?, f: FunC[Vertex?, AnyC]) -> NoneC

  def dfs_to_list(graph: WUGRAPH!, start: Vertex?) -> VertexList?
  def one_of(x: AnyC, vec: VecC[AnyC]) -> bool?

```

project-lib/binheap.rkt

```

interface PRIORITY_QUEUE[X]:
  def len(self) -> nat?
  def find_min(self) -> X
  def remove_min(self) -> NoneC
  def insert(self, element: X) -> NoneC

class BinHeap[X] (PRIORITY_QUEUE):
  def __init__(self, capacity, lt?)

  def heap_sort[X](v: VecC[X], lt?: FunC[X, X, bool?]) -> NoneC

```